

Fletch: File-System Metadata Caching in Programmable Switches

Qingxiu Liu, Jiazhen Cai, Siyuan Sheng, Yuhui Chen, Lu Tang, Zhirong Shen, and Patrick P. C. Lee

Abstract—Fast and scalable metadata management across multiple metadata servers is crucial for distributed file systems to handle numerous files and directories. Client-side caching of frequently accessed metadata can mitigate server loads, but it incurs significant overhead and complexity in maintaining cache consistency when the number of clients increases. We propose Fletch, an in-switch file-system metadata caching framework that leverages programmable switches to serve file-system metadata requests from multiple clients directly in the switch data plane. Unlike prior in-switch key-value caching systems, Fletch addresses file-system-specific path dependencies under stringent switch resource constraints. We implement Fletch atop HDFS and evaluate it on a Tofino-switch testbed using real-world file-system metadata workloads. Fletch achieves up to 181.6% higher throughput than vanilla HDFS and complements client-side caching with throughput gains of up to 139.6% on 128 simulated metadata servers.

Index Terms—Programmable switches, distributed file systems, file-system metadata caching.

I. INTRODUCTION

Scaling file-system metadata management across multiple metadata servers is crucial for distributed file systems to handle billions of files and directories. Field studies, both classical [29], [50] and recent [58], indicate that metadata operations dominate file-system requests. For example, 67-96% of file-system requests in Baidu AI Cloud are metadata-related [58]. As the number of files and directories grows, especially in workloads dominated by small files [18], [28], metadata operations, such as inode lookups, permission checks, and directory traversals, become performance bottlenecks. File-system metadata access patterns in practice tend to be highly skewed, with a small fraction of files and directories accessed far more frequently than others [13], [15], [36]; for example, less than 3% of files account for 34-39% of requests in Yahoo’s HDFS clusters [13]. Metadata servers are frequently bottlenecked by skewed access, leading to severe load imbalance where the entire cluster’s performance is limited by a few overloaded servers.

Caching frequently accessed metadata on the client side is a common strategy to mitigate loads on metadata servers. While client-side caching distributes the caching load across clients,

it introduces a severe distributed cache consistency problem: maintaining consistency across numerous clients requires complex, high-overhead lease management or invalidation protocols [39], [44], [49], [60], since any metadata update must notify all relevant clients. Maintaining this consistent state requires metadata servers to track every participant’s cache and broadcast invalidations upon updates, resulting in a communication complexity of $O(N)$, where N is the number of clients. This overhead incurs high latency and makes client-side caching increasingly fragile in dynamic environments with frequent client churn. More fundamentally, because client-side caches are distributed across individual clients, they cannot coordinate to resolve the severe load imbalance at the backend servers caused by highly skewed file-system access patterns. Existing client-side caching approaches [39], [44], [49], [60] further limit their scope to caching directory permission metadata to mitigate path resolution overhead (§II-A) while still relying on metadata servers for attribute retrieval, so their overall performance gains remain modest.

Programmable switches [17] offer a compelling and unique alternative by leveraging their fundamentally centralized network position to enable *in-switch caching*. Since the switch sits on the critical path between all clients and servers, it maintains a global view of all metadata requests, a structural vantage point that effectively addresses the limitations of client-side caching. First, since cache updates are applied directly within the network rather than coordinated across distributed clients, in-switch caching natively eliminates the need for inter-client invalidation protocols, reducing the complexity of cache consistency maintenance from $O(N)$ to $O(1)$. Second, in-switch caching absorbs highly skewed, hot-path requests directly in the switch data plane, thereby effectively shielding backend servers from load imbalance.

While prior work has explored in-switch caching for key-value stores [26], [32], [35], [38], [40], [43], [52], in-switch caching for file-system metadata poses unique challenges that have not been directly addressed before. First, file-system pathnames, unlike fixed-length keys, have large, variable sizes and cannot readily fit within limited switch resources. Second, accessing a file’s metadata requires traversing the metadata of its internal directories, thereby exacerbating switch resource demands when caching multiple levels of metadata. Third, cache lookups for file and directory metadata require multiple iterations under the strict switch programming model, complicating concurrent cache updates and lookups while maintaining consistency and performance.

We propose Fletch, an in-switch file-system metadata caching framework tailored for distributed file systems under skewed,

This work was supported in part by National Natural Science Foundation of China (62302410, U25B2037) and Research Grants Council of Hong Kong (GRF 14201523). Corresponding author: Patrick P. C. Lee.

Qingxiu Liu, Jiazhen Cai, Siyuan Sheng, and Patrick P. C. Lee are with the Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong (e-mail: qxliu23@cse.cuhk.edu.hk; jzcaic@cse.cuhk.edu.hk; siyuanshengpro@gmail.com; pcleec@cse.cuhk.edu.hk).

Yuhui Chen, Lu Tang and Zhirong Shen are with the Department of Computer Science and Technology, Xiamen University, Xiamen 361005, China (e-mail: yhchen230@stu.xmu.edu.cn; tanglu@xmu.edu.cn; zhirong.shen2601@gmail.com).

read-intensive workloads. Fletch extends in-switch key-value caches by specifically addressing file-system path dependencies through various techniques: (i) *path-aware cache management* to account for path dependencies in cache admission and eviction; (ii) *multi-level read-write locking* to enable high-performance concurrent cache lookups and updates; and (iii) *local hash collision resolution* to efficiently and correctly map variable-length paths into fixed-length keys. It targets highly skewed, read-intensive file-system workloads and supports two deployment modes (§III-C), a *standalone* mode (replacing client-side caching, eliminating $O(N)$ consistency overhead) and a *complementary* mode (augmenting client-side caching with in-switch load balancing), for distinct deployment regimes.

We implemented Fletch in P4 [16] and compiled it into the Tofino switch chipset [9], [10]. We integrated Fletch with Hadoop HDFS [5], while preserving HDFS semantics. We also implemented a state-of-the-art client-side caching approach [44], [49] to demonstrate how Fletch complements it and further improves file-system metadata access performance. Our evaluation on four real-world workloads [44], [49], [62] shows that with 128 simulated metadata servers, Fletch achieves up to 181.6% higher throughput than vanilla HDFS, while client-side caching coupled with Fletch achieves up to 139.6% higher throughput than without Fletch. We release the source code of Fletch at <https://github.com/adslabcuhk/fletch>.

II. BACKGROUND AND MOTIVATION

A. File-System Metadata Management

File systems organize data in a hierarchical, tree-based namespace, where files and directories are managed as leaf and non-leaf nodes, respectively. Each node is identified by a *path* and contains *metadata* (e.g., owner, size, and permission). A path often has multiple internal directories. We refer to a file or directory below the root as the i -th *level* ($i \geq 1$), and the maximum number of levels of a path as the *depth*. For example, the path `/a/b/c.txt` has a depth of 3, with levels `/a`, `/a/b`, and `/a/b/c.txt`. In the namespace, we say that path p is an *ancestor* of path q , and q is a *descendant* of p , if p is an internal directory (or prefix) of q . We say that p is the *parent* of q , and q is a *child* of p , if p is an ancestor of q with exactly one less level than q (e.g., `/a` is the parent of `/a/b`, and `/a/b` is a child of `/a`).

File-system metadata management is crucial for distributed file systems. There are various metadata operations for files (e.g., `create`, `delete`, `open`, and `close`), directories (e.g., `mkdir` and `rmdir`), attributes (e.g., `chmod`, `chown`, and `utime`), and data organization (i.e., `rename`, `readdir`, and `stat`). File-system metadata operations often rely on *path resolution*, which parses and traverses each level of a path to verify metadata for existence and permissions.

B. Programmable Switches

Programmable switches [17] enable customized packet processing tasks beyond traditional packet forwarding. A programmable switch comprises a data plane and a control plane. The data plane, as shown in Figure 1, performs packet forwarding via multiple *ingress* and *egress* pipelines. Packets

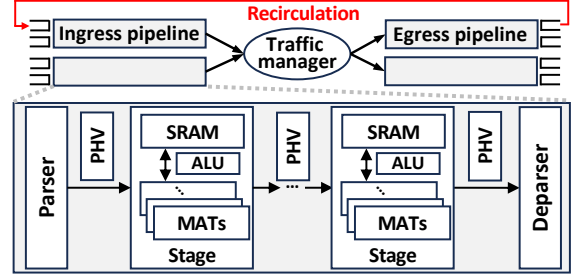


Figure 1: Data plane of a programmable switch.

enter an ingress pipeline and are forwarded to an egress pipeline via a *traffic manager*. The control plane manages the data plane by specifying packet processing rules.

Each ingress or egress pipeline processes packets via a series of *stages*, each of which contains *match-action tables* (MATs) that execute processing logic based on the Reconfigurable Match-Action Table (RMT) paradigm [17]. A *parser* converts packet headers into *packet header vectors* (PHVs), processed by MATs with on-chip ALUs across stages. Each stage can only access limited SRAM, typically with tens of memory blocks. The switch programming model imposes strict constraints: each memory block can be accessed at most once per PHV traversal, and stages cannot access memory blocks of other stages. After processing, a *deparser* reconstructs packets. To process packets in multiple iterations, switches support *recirculation*, which redirects packets from an egress pipeline back to an ingress pipeline. Recirculation should be cautiously used, as it consumes extra switch resources and slows down packet forwarding. In this work, we leverage recirculation for read requests that require in-switch path resolution (§IV-A) and write requests that await locks (§V-B).

C. Challenges

Designing in-switch metadata caching for distributed file systems poses the following challenges.

Challenge 1: Constrained switch resources. Switches have limited resources and complicate caching implementation. For example, Tofino switches [10] provide only 12 stages per ingress or egress pipeline, each processing up to 16 bytes due to ALU word size restrictions. The PHV size is capped (e.g., 768 bytes in Tofino switches [10]), and all pipelines share limited SRAM. For comparison, NetCache [32] allocates SRAM for access frequency tracking and supports only a maximum value size of 128 bytes for key-value records.

File-system metadata caching is particularly resource-intensive. HDFS file or directory names can reach 255 bytes [4] and aggravate SRAM and PHV overhead. Path resolution for a path (e.g., `/a/b.txt`) requires accessing metadata for its ancestors (i.e., `/` and `/a`) (§II-A). Caching metadata for all paths and their ancestors is desirable but challenging given limited SRAM. Splitting path processing across multiple levels requires careful synchronization across stages. To mitigate switch resource usage, existing in-switch key-value caches [32], [43], [52] offload cache admission and eviction to a centralized controller, but excessive switch-to-controller communications incur high latencies.

Challenge 2: Cache consistency. Enforcing cache consistency under concurrent updates is crucial, as programmable switches handle requests from multiple ingress pipelines and trigger simultaneous cache updates. Cache updates may overlap with path resolution, thereby further complicating cache consistency management. For example, a read request for `/a/b.txt` requires metadata access for `/`, `/a`, and `/a/b.txt`, while concurrent `chmod` requests for `/a` and `/a/b.txt` update their metadata and may occur between their cache lookups. Thus, the read request can access a mix of pre- and post-updated metadata, leading to inconsistencies.

III. DESIGN OVERVIEW

A. Goals and Assumptions

Fletch is an in-switch metadata caching framework for distributed file systems, aiming to achieve high throughput and low latencies for file-system metadata requests across multiple metadata servers. It targets read-intensive file-system metadata workloads, as observed in prior studies [44], [49], [62]. For example, in a LinkedIn HDFS cluster, 84% of 145 million metadata operations are lookups, with only 9% creates and 7% updates [49]; in Alibaba’s Pangu file system, over 60% of metadata operations are reads [44].

To ensure consistency, Fletch adopts *write-through* caching, which synchronously updates both the in-switch cache and server-side metadata before acknowledging clients, as in prior in-switch key-value caches [32], [40], [42], [43], rather than write-back caching. While write-back caching reduces write latency by buffering updates in the switch data plane and immediately acknowledging clients, this benefit is marginal for read-intensive file-system metadata workloads, which Fletch targets, since writes constitute only a small fraction of all requests, and the aggregate impact of high per-write latency on overall throughput is limited. Most importantly, under write-back caching, the switch data plane holds the sole up-to-date copy of new metadata until it is written back to the server; if the switch fails before this write-back completes, the new metadata is lost. Write-through caching eliminates this risk by ensuring that metadata servers always hold the latest committed state, providing durability independent of switch failures.

Since path resolution always starts at the root, Fletch further assumes that the root directory’s metadata is persistently cached, with its permissions unchanged and the root never deleted [46], consistent with standard distributed file-system practices.

B. Architecture and Design Roadmap

Figure 2 depicts Fletch’s architecture. In the control plane, a controller manages in-switch cache admission and eviction, while in the data plane, multiple clients send (receive) file-system metadata requests (responses) via the switch to (from) multiple metadata servers (or servers for short). Fletch employs three techniques to enable efficient in-switch file-system metadata caching.

Path-aware cache management (§IV). Fletch caches file-system metadata in the switch data plane by hashing variable-length paths into fixed-length keys and managing file-system metadata in key-value pairs as in existing in-switch key-value

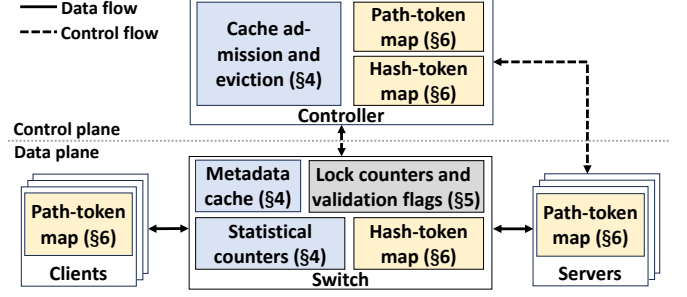


Figure 2: Fletch’s architecture.

caches [32], [40], [43], [52]. Unlike in-switch key-value caches, Fletch addresses path dependencies across hierarchical levels during cache admission and eviction.

Multi-level read-write locking (§V). To synchronize concurrent metadata operations, Fletch adopts multi-level read-write locking using (i) lock counters as read-write locks for different path levels and (ii) validation flags for verifying the validity of cached paths. It ensures reliable locking and unlocking during path resolution, even in unreliable networks.

Local hash collision resolution (§VI). Mapping paths to fixed-size keys in the switch data plane can cause hash collisions, leading to incorrect metadata retrieval. For example, if two paths `/a` and `/b` share the same hash and `/a` is cached, a request for `/b` can erroneously return `/a`’s metadata. Fletch proposes a *local* hash collision resolution approach, where the controller assigns unique values, called *tokens*, to hash-colliding paths and distributes the tokens to clients, servers, and the switch, so as to allow local resolution of hash collisions without the controller’s intervention for every request. Fletch ensures no incorrect metadata retrieval.

C. Deployment Modes

Fletch supports two deployment modes that share an identical switch-side design: for a read request, the switch performs path resolution against its in-switch cache and serves the request directly on a cache hit; on a cache miss, it forwards the request to the corresponding server. All write requests are forwarded under write-through caching (§III-A). The two modes differ only in client-side configuration and server-side processing.

The *standalone* mode targets deployments where client-side caching is undesirable (e.g., under frequent client churn or large client populations). Clients have no local metadata cache and forward requests directly to the switch; the server processes forwarded requests as in vanilla distributed file systems.

The *complementary* mode targets deployments where client-side caching is already in place but cannot resolve server-side load imbalance under skewed access. Clients additionally cache directory permission metadata locally [44], [49], resolve directory permissions from their local cache, and forward each request to the switch together with the local permission check results and the versions of the cached permissions used. The server processes the request based on its own authoritative state; if the attached permission versions are stale or absent, the server additionally returns the latest permission metadata for the client to refresh its local cache. We evaluate Fletch (standalone) and Fletch+ (complementary) in §IX.

IV. PATH-AWARE CACHE MANAGEMENT

A. Path Representation

Fletch incorporates path awareness into cache admission and eviction by caching frequently accessed paths and their ancestors, so as to mitigate cache misses during path resolution. This ensures that if a path is cached, its ancestors are also cached, thereby allowing the reuse of cached metadata across different paths with the common ancestors.

Fletch maps a file-system pathname into fixed-size hash keys for efficient switch operations. For a read operation, a Fletch client partitions a path into multiple levels (including the root) and computes a hash for each level. For example, `/a/b/c.txt` is partitioned into `/`, `/a`, `/a/b`, and `/a/b/c.txt`, with each level being hashed. To avoid redundant computations, the hash of the root directory `/` is pre-computed and cached on the client side. For a write operation, the client computes a hash for the complete path without partitioning, as all writes are forwarded to the server (that holds the namespace) for path resolution under write-through caching (§III-A). Currently, Fletch uses the first 64 bits of a 128-bit MD5 hash as the hash value, where MD5 is used for fast hashing. Collision resistance is addressed separately via tokens (§VI).

Fletch caches file-system metadata as key-value records in the switch. Each record is identified by its hash key, and its value contains file or directory metadata. Files and directories have common metadata fields (i.e., type, permissions, owner/group IDs, and timestamps); while files additionally include the size and replication factor fields. Each file's metadata has 40 bytes, and each directory's has 24 bytes.

Fletch manages cache across multiple stages in the switch. It leverages MATs across two stages in a single ingress pipeline to perform cache lookups using hash keys, thereby avoiding cross-pipeline synchronization (§V) and enforcing logical dependency (i.e., cache hit/miss status is resolved before metadata retrieval can proceed). Since packets are strictly processed in ingress-to-egress order, this placement enables fast cache status checks. As essential cache status checking and other pipeline operations consume all available stages in the ingress pipeline, the switch allocates 32 register arrays across eight egress stages to store cached metadata values (§VIII). To simplify routing, the switch caches metadata in the egress pipeline that is connected to the corresponding server.

Clients issue metadata requests, including hash keys and full pathnames, to the switch, which reports frequently accessed pathnames to the controller during cache admission (§IV-B). For reads, the switch performs path resolution by issuing cache lookups and permission checks for each level, starting from the root, and recirculates the request to the ingress pipeline for each next level. Due to limited stages and memory access constraints (§II-B), path resolution across multiple levels cannot be completed in a single pass but instead requires one recirculation per level; the corresponding overhead is quantified in §IX-B. For a cache hit (i.e., the metadata for the path and all its ancestors are cached), the switch returns the metadata if all levels' permission checks pass, or an error if any check fails. To serve cache hits directly from the egress pipeline without an additional egress-to-ingress recirculation round-trip, Fletch uses

packet cloning: the switch clones the in-flight request, embeds the retrieved metadata into the clone, rewrites the clone's destination to the originating client, and drops the original packet. This allows the traffic manager to forward the response immediately. For a cache miss, the switch forwards the request to the corresponding server. The switch also reports frequently accessed pathnames to the controller to drive cache admission decisions (§IV-B). Since writes require multi-level locking to ensure consistency, their processing differs fundamentally from reads; the detailed write workflow is deferred to §V.

B. Cache Admission and Eviction Workflows

Data structures. Fletch monitors path access frequencies for read operations (excluding access to ancestors during path resolution). For uncached paths, which dominate path access traffic, Fletch uses a Count-Min Sketch (CMS) [23], as in prior studies [32], [43], [52], to estimate their access frequencies within fixed-size memory with provable error bounds. For cached paths, Fletch tracks their exact access frequencies using a frequency counter array. The switch periodically reports the access frequencies of all cached paths to the controller for eviction decisions, and resets the CMS and frequency counter array after each reporting.

Cache admission. Fletch's cache admission is triggered by the switch data plane. During runtime, the switch monitors the access frequencies of uncached paths via the CMS and identifies *hot* paths (i.e., those exceeding a pre-defined CMS threshold) for admission. When a hot path p is detected, the switch notifies the controller, which retrieves the metadata for p and its uncached ancestors from the servers. The controller communicates with servers using UDP (and issues retransmissions if needed), bypassing the switch data plane to avoid critical-path overhead, as in prior work [32], [52].

The controller verifies cache capacity. If the cache is not full, the controller sends the hash keys and metadata for p and its uncached ancestors to the switch for admission (note that the controller maintains a global view of all cached paths). Unlike NetCache [32] (without path awareness), Fletch admits metadata for both p and its uncached ancestors to ensure path-aware caching. To ensure cache consistency during cache admission, while the controller is adding a path to the cache, write requests to this path are blocked at the server until the server is notified that cache admission is finished [32]. Since file-system metadata workloads are read-intensive (§III-A), the blocking action has limited performance impact.

Cache eviction. When the cache is full and an uncached hot path p is reported, the controller triggers cache eviction to reclaim cache space for p and its uncached ancestors. It selects cached path candidates for eviction based on periodically reported access frequencies, prioritizing the least frequently accessed path with no cached descendants. If the selected path is the only cached child of its parent, the parent is also selected. The controller recursively includes ancestors as candidates until reaching an ancestor with multiple cached children or the root. The rationale of this recursive inclusion is that the selected ancestors are unlikely to be accessed by themselves, as directory operations are infrequent in practice [44], [49], [62] (e.g., only

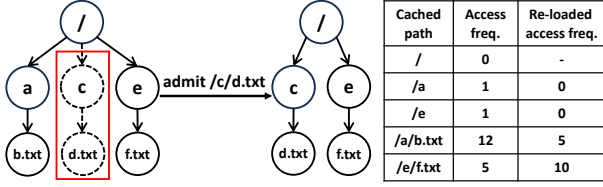


Figure 3: Example of cache admission and eviction workflows.

4.2% of metadata operations in Alibaba’s Pangu file system are directory-related [44]). By reclaiming ancestral slots only when they no longer support any cached descendants, Fletch ensures path resolution integrity and maximizes SRAM utilization.

The controller selects candidates up to a pre-defined threshold, currently set as $2\times$ the number of paths to be admitted (i.e., p and its uncached ancestors). A larger threshold reduces the risk of accidentally evicting recently active paths, since the controller’s view of access frequencies may be slightly stale between reporting intervals; however, it increases the number of control-plane read packets issued to the switch. In practice, Fletch is robust to this parameter in both directions. A threshold slightly below $2\times$ has a negligible impact on throughput: any mistakenly evicted path is rapidly re-admitted in the subsequent reporting interval (as seen from Exp#8 in §IX-C under dynamic workloads); a threshold above $2\times$ incurs only marginal additional overhead: for a path depth of nine (our evaluation default; §IX-A), even a $3\times$ threshold introduces at most nine additional read packets over the $2\times$ default, negligible compared to the round-trip latency of cross-node synchronization and metadata retrieval incurred per admission. Thus, Fletch adopts $2\times$ as the default.

The controller reloads the current access frequencies of the selected candidates from the switch and evicts the least frequently accessed path with no cached descendants, along with any ancestors having only one cached child. It repeats the selection until sufficient cache space is reclaimed. Finally, the controller notifies the switch to evict the selected paths and admits p and its uncached ancestors.

Example. Figure 3 depicts the cache admission and eviction workflows. Consider a full cache with five records, holding $/$, $/a$, $/e$, $/a/b.txt$, and $/e/f.txt$ with access frequencies 0, 1, 1, 12, and 5, respectively, where $/$ is always cached. Suppose that the switch reports an uncached hot path $/c/d.txt$, with a frequency 10, to the controller. As $/c$ is uncached, the controller aims to admit both $/c$ and $/c/d.txt$. With a full cache, the controller triggers cache eviction to select four candidates (i.e., $/a$, $/e$, $/a/b.txt$, and $/e/f.txt$), twice the number of paths to be admitted, and reloads their current frequencies (0, 0, 5, and 10, respectively). The path $/a/b.txt$ has the lowest access frequency among all paths without cached descendants, and its ancestor $/a$ (with only one cached child) will also be evicted. The controller notifies the switch to evict $/a$ and $/a/b.txt$, and admits $/c$ and $/c/d.txt$.

V. IN-SWITCH READ-WRITE LOCKING

A. Lock Design

Fletch adopts multi-level read-write locking to ensure cache consistency under concurrent metadata updates. The locking

mechanism operates entirely within the switch data plane to avoid controller overhead. It employs multiple *lock counter arrays* and a *validation array*, both implemented as register arrays in the switch data plane and initialized with zero entries. Each cached path is associated with one lock counter (a slot in a lock counter array) and one validation flag (a slot in the validation array), indexed by the path’s hash key.

Lock counter arrays. Fletch assigns one lock counter array to each of eight path-level groups: the first seven arrays serve levels 1 through 7 individually (e.g., the level-1 array covers paths of the form $/a$; §II-A), and the eighth array serves all paths at depth 8 and beyond, keyed by their level-8 hash component. Each array contains 65,536 counters, indexed by the last 16 bits of a path’s hash key; each counter records the number of active read requests for the paths mapped to that slot. A cached path is associated with its array by level and with a specific counter by the last 16 bits of its hash key. Concentrating dedicated arrays on the shallowest levels is justified by workload evidence: 90% of accessed paths have a depth of no more than 10 [14], [45], with some workloads approaching 100% [25]. With 65,536 counters per array, the probability that two distinct paths at the same level share a slot is $1/65,536$, keeping both slot collisions and lock contention low. Each counter is 16 bits wide, supporting up to 65,535 concurrent read requests per slot to sustain high read parallelism. In case of a slot collision (i.e., two or more paths share the same last-16-bit index at the same level), the counter tracks the aggregate read count across all colliding paths, implying that a write to any one of the colliding paths must wait until all reads at that slot complete; however, the low slot-collision probability makes this overhead negligible in practice.

Validation array. The validation array [32], [52] tracks metadata validity for all cached paths. A validation flag of one indicates valid metadata and allows reads from the cache, while a flag of zero indicates invalid or updating metadata and all reads are directed to servers. After a cache update, its validation flag is set to one to permit subsequent reads.

B. Read and Write Flows

Fletch places the validation array across all egress pipelines, co-located with value register arrays (§IV-A) for efficient validity checks. Lock counter arrays reside in a single ingress pipeline to avoid cross-pipeline synchronization. Fletch redirects requests arriving at other ingress pipelines to the ingress pipeline holding lock counter arrays via cross-pipeline recirculation (§IX-A). This placement is critical, as lock counter arrays should be positioned before the validation array to ensure correctness, and they cannot be placed in egress pipelines due to insufficient switch resources. In Fletch’s deployment, the switch data plane is not a bottleneck, and consistent packet processing ordering is maintained across ingress and egress pipelines, so Fletch ensures efficient and correct locking and validation operations.

Reads. Reads are classified as (i) *single-path reads* (e.g., `stat`), which retrieve only metadata of the requested path; and (ii) *multi-path reads* (e.g., `readdir`), which retrieve metadata of the requested path and its descendants. Single-path reads are served

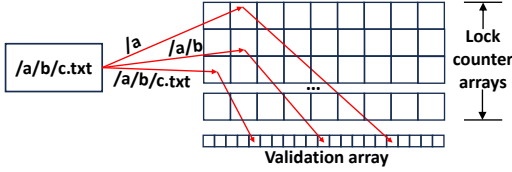


Figure 4: Example of processing a read request under multi-level read-write locking.

from the in-switch cache, while multi-path reads are forwarded to servers to ensure correctness, as descendant paths may be uncached and servers maintain the authoritative namespace to resolve partially caching scenarios. Multi-path reads are rare in practice (e.g., only 3.9% in Alibaba’s workloads [44]; see Table II). Upon receiving a (single-path) read request, the switch checks if the path’s last level is cached, implying that all its ancestors are also cached (§IV). If so, the switch increments the lock counter for each path level by one and resolves the path via recirculation.

During path resolution, the switch checks the validation flag for each level. If the validation flag is one (i.e., valid metadata and no ongoing write), the switch retrieves the metadata from cache and performs permission checks. If the permission checks pass, the switch proceeds to the next level and decrements the lock counter for the previous level by one. After resolving the whole path, the switch decrements the lock counter for the last level by one via one extra recirculation, ensuring that all lock counters are released. If permission checks fail at any level, the switch sends an error response to the client and decrements the lock counters from the failure point to the requested path by one. Conversely, if the validation flag is zero (i.e., invalid or updating metadata), the switch forwards the read request to the server, which returns a response. The switch then decrements all lock counters from the invalid metadata point to the requested path by one, and returns an ACK to the server. If the server does not receive the ACK before timeout, it retransmits the same response.

For example, consider three cached paths `/a`, `/a/b`, and `/a/b/c.txt` (Figure 4). For a read request open `/a/b/c.txt`, the switch maps each path level `/a`, `/a/b`, and `/a/b/c.txt` to a lock counter in the first, second, and third lock counter arrays, respectively, with corresponding validation flags. The switch increments the lock counters for all levels by one. If `/a`’s validation flag is one and its permission check passes, it processes `/a/b` and decrements `/a`’s lock counter by one. If `/a/b`’s validation flag is zero, it forwards the request to the server, which retrieves `/a/b/c.txt`’s metadata. The server returns a response, which triggers the switch to decrement the lock counters for both `/a/b` and `/a/b/c.txt` by one (assuming no packet loss). The switch then returns an ACK to the server. **Writes.** Writes are classified as (i) *single-path writes* (e.g., `chmod` and `chown`), which update only the requested path, and (ii) *multi-path writes* (e.g., `chmod -R` and `chown -R`), which update the requested path and its descendants. The locking mechanism ensures consistency for both.

(i) *Single-path writes.* For a single-path write, the switch checks if the path is cached. If so, it checks the corresponding lock counter based on its level and hash key. If the lock

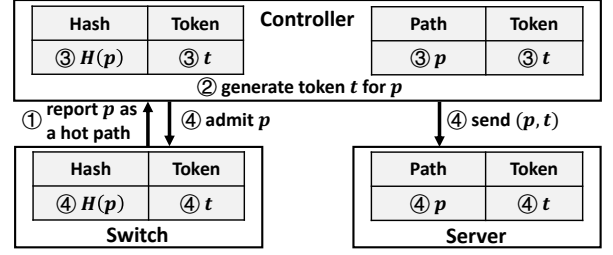


Figure 5: Example of token generation and distribution. Note that the controller also assigns tokens for p ’s uncached ancestors in Steps 2-4. We omit the details for brevity.

counter is non-zero (i.e., with ongoing reads), the switch recirculates the request until the lock counter reaches zero (i.e., no ongoing read). Then, the switch sets the validation flag to zero and forwards the request to the server. If the write is successfully completed, the server returns a response, which triggers the switch to update the cached path’s metadata and set the validation flag to one; otherwise, only the validation flag is set to one without cache updates.

The current reader-preferring lock design is susceptible to writer starvation: while a write request recirculates waiting for the lock counter to reach zero, new incoming reads increment the counter before the write can acquire the lock. Under a continuous stream of read requests, the counter never reaches zero, blocking the write indefinitely. A potential remedy is to block new reads once a pending write is detected at the switch (e.g., by setting a write-pending flag that arriving reads must check before incrementing the counter [64]). We acknowledge writer starvation as a current limitation and leave the design of a starvation-free locking mechanism as future work.

(ii) *Multi-path writes.* Multi-path writes follow the same single-path write processing until the request reaches the server. If the write is successfully completed, the server updates the cache for all cached descendants before the requested path, so that the requested path remains invalidated until all cached descendants are fully updated. By performing path resolution in a top-down manner, Fletch prevents reads from accessing cached descendants before the requested path is updated (i.e., until the cache updates for a multi-path write are completed) to ensure cache consistency.

VI. LOCAL HASH COLLISION RESOLUTION

Fletch maps file-system pathnames to 64-bit hash keys (§IV-A), so hash collisions are possible (albeit unlikely) and lead to incorrect metadata retrieval. While the controller holds a global view of all cached paths (§IV-B), querying the controller to resolve hash collisions for every request incurs high switch-to-controller latencies (§II-C). Fletch adopts a local, token-based hash collision resolution mechanism by synchronizing the controller’s global view of cached paths with the switch, clients, and servers, preserving correctness and high performance.

A. Token-Based Hash Collision Resolution

Token definition and structures. A *token* is an 8-bit value paired with a 64-bit hash key to uniquely identify a path. *Valid* tokens range from 1 to 255, while 0 indicates *invalid*. A cached

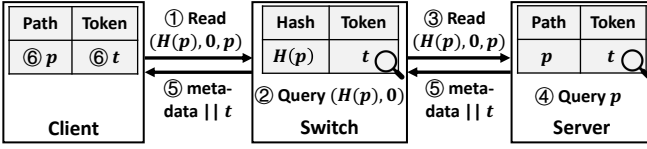


Figure 6: Example of how a client updates its path-token map. Note that the client also attaches tokens of p 's ancestors in Step 1, the server returns the tokens for p 's ancestors in Step 5, and the client adds the tokens in Step 6. We omit the details for brevity.

path is assigned token 1 if no collision occurs, or the next available token (e.g., 2) if it collides with an existing cached path. Fletch maintains two unordered map structures: (i) a *path-token map*, which records paths and their tokens (*path-token pairs*), and (ii) a *hash-token map*, which records hash keys and their tokens (*hash-token pairs*). The controller keeps both maps, each client and server holds a path-token map, and the switch holds a hash-token map. All maps are kept in memory for fast lookup. For failure recovery (§VII-C), the controller additionally maintains two persistent logs in a local key-value store (e.g., RocksDB [12]), each recording a snapshot of both maps: (i) an *active log* covering paths currently in the cache, used to repopulate the switch cache and hash-token map after a switch failure and to restore a recovered server's path-token map; and (ii) a *historical log* covering all paths ever cached, used to recover the controller's own maps upon restart.

Token generation. During cache admission, the controller assigns tokens to each level of a hot path, as shown in Figure 5. When the switch reports a hot path p for admission (① in Figure 5), the controller checks its map structures. If p is the first time being admitted and its hash key is unique, the controller assigns a token of value one; if a collision occurs, it assigns the next available token (e.g., t) (② in Figure 5). The controller updates its path-token and hash-token maps with the new path-token and hash-token pairs, respectively (③ in Figure 5). These entries persist in the controller's maps even after eviction: if an evicted path is later re-admitted, its previously assigned token is reused, ensuring consistent token identity across admission cycles. To support failure recovery, cache admission appends the new entry to both the active and historical logs, while cache eviction removes the entry only from the active log, with the historical record preserved.

Token distribution. During cache admission, the controller distributes each admitted path and its token to the switch and the relevant server that holds the path (④ in Figure 5). The switch adds the hash key and token to the hash-token map, while the server adds the full path and token to the path-token map. These entries are removed during cache eviction, as notified by the controller.

Token discovery and utilization. Clients update their path-token maps when issuing read or write requests, as shown in Figure 6. Initially, with an empty path-token map, a client attaches an invalid token (value zero) to a request (e.g., reading path p 's metadata) sent to the switch (① in Figure 6). The switch detects a cache miss by querying its hash-token map (② in Figure 6) and forwards the request to the server (③ in Figure 6). The server retrieves the metadata, checks its path-token map (④ in Figure 6), and responds with the metadata and

valid tokens (⑤ in Figure 6). The client updates its path-token map with the valid tokens for future requests (⑥ in Figure 6). For cached paths, the switch confirms cache hits when valid tokens are included in the request.

B. Token Scalability and Overhead Analysis

Scalability analysis. Fletch's scalability depends on two properties: its ability to tolerate hash collisions among cached paths, and its ability to store mapping entries across a growing namespace without unbounded resource consumption.

For collision tolerance, Fletch uses 8-bit tokens to distinguish up to 255 distinct paths that map to the same 64-bit hash index. For 64-bit hash keys, the pairwise collision probability between any two specific paths is $1/2^{64}$. Given a cache of C entries, the expected number of cached paths sharing any particular hash index is $C/2^{64}$, which is close to zero even for billion-scale namespaces. The probability that any single hash index accumulates more than 255 cached paths is hence extremely small, and 8-bit tokens provide sufficient collision capacity for any practical file-system deployment.

For storage scalability, the controller's maps grow monotonically: since token assignments persist in the controller's maps even after cache eviction (§IV-B), re-admitted paths can reuse their previously assigned tokens. In the worst case, the controller accumulates entries for all distinct paths that have ever been cached. Since the controller persists its maps to a local key-value store for failure recovery (§VI-A), the maps can scale beyond DRAM capacity by leveraging the key-value store's on-disk storage. In contrast, storage requirements for the switch and servers are strictly bounded by the current cache size, since entries are installed upon admission and removed upon eviction, so the active state in the switch data plane and on each server scales with the cache size rather than the total namespace. Client storage is bounded by the active working set: clients attach an expiry time (e.g., one hour) to each local path-token entry and periodically clear stale entries, preventing indefinite accumulation.

Overhead analysis. Fletch's token mechanism incurs minimal communication and storage overhead. For each request, a client attaches one (hash, token) pair per path level (including the root), contributing 9 bytes per level (an 8-byte hash key and a 1-byte token). The total per-request PHV overhead for a path of depth d is $9(d+1)$ bytes, much smaller than the native HDFS representation, where each path component can reach 255 bytes (§II-C). This compact per-level encoding substantially alleviates pressure on the switch's limited PHV capacity (768 bytes on Tofino). Although per-request PHV overhead grows linearly with path depth, it remains practically small because real-world workloads exhibit small path depths (§V-A; for example, 90% of accessed paths have a depth of no more than 10).

The storage overhead of the token mechanism is small across all components. In the switch, the hash-token map stores 9-byte entries (an 8-byte hash key and a 1-byte token); it is implemented using MATs rather than register arrays to enable efficient exact-match lookups without the entry-size restrictions of SRAM registers. The controller maintains a hash-token map covering all paths ever cached rather than only currently cached

ones, alongside its path-token map; the path-token map is also held by servers and clients. For these path-token maps, each entry pairs the full path string with a 1-byte token. Since path strings are inherent to the file-system namespace and are always stored, the extra storage cost incurred by the token mechanism is at most 1 byte per cached path across all components.

VII. FAILURE ANALYSIS

We present Fletch’s complete failure-handling design across clients, the switch, the controller, and servers, covering cache admission (§VII-A), request serving (§VII-B), and recovery (§VII-C). A few mechanisms are described as extensions beyond our current prototype (§VIII).

A. Failures during Cache Admission

Cache admission is orchestrated among the switch, controller, and servers (§IV-B), so Fletch must remain resilient to both packet loss and component failures during this process. Note that server failures are orthogonal to cache admission correctness: since the cache state and management logic reside exclusively in the controller and switch, a server failure does not corrupt the global cache state. Server-level fault tolerance during cache admission is delegated to the backend distributed file system, which provides replication and recovery, ensuring that the controller can always retrieve metadata for a path under admission from a surviving replica.

Packet loss. Fletch ensures end-to-end reliability for the admission protocol through proactive redundancy and ACK-based retransmission. Communication between the controller and servers uses UDP with a timeout-and-retry mechanism to guarantee reliable delivery of both metadata fetches and cache-completion notifications. On the switch-to-controller channel, the switch proactively transmits each hot-path report and admission acknowledgment multiple times (three by default) to compensate for data-plane packet loss [32]. On the controller-to-switch channel, the controller reissues a cache update packet if no acknowledgment is received within a configured timeout. These mechanisms ensure that the admission handshake always terminates, preventing writes from being indefinitely blocked or leaving paths in an inconsistent intermediate state.

Switch and controller failures. Recall that write requests to the path undergoing admission are held at the server until the server receives a cache-completion notification from the controller, which is triggered only after the switch acknowledges the cache update (§IV-B). Thus, a failure of either the switch or the controller can stall the admission chain, blocking the write indefinitely. To prevent this, Fletch employs a server-side timeout mechanism: if the server does not receive the cache-completion notification within a preconfigured window, it lifts write blocking, registers an admission failure, and resumes normal request handling. This ensures that a failure in either the data plane or the control plane does not permanently impair the backend file system’s availability.

B. Failures during Request Serving

Packet loss. A lost switch-to-server request or server-to-switch response causes the client to time out and retransmit, ensuring

eventual delivery. However, a subtle correctness issue arises when a retransmitted read targets a path whose validation flag is currently zero (i.e., being updated by a concurrent write). In this case, the switch increments the affected lock counter without a corresponding decrement from the earlier lost exchange, thereby permanently elevating the counter and blocking subsequent writes to that path. Fletch addresses this by having the controller periodically monitor lock counters and reset stale ones.

The controller distinguishes a stale counter from a legitimate in-flight read by observing that any in-flight read continually drives counter changes: each request, retransmission, or response triggers an increment or decrement. The maximum interval between two consecutive such events is bounded by T_{event} , determined by the client retransmission timeout, network round-trip time, and server processing time (typically hundreds of milliseconds). The controller sets its detection window $T_{\text{stale}} \gg T_{\text{event}}$ (e.g., $T_{\text{stale}} = 30$ s), so that any counter unchanged for longer than T_{stale} cannot be referenced by a live request and is safely classified as stale.

Once a stale counter is identified, the controller records its observed value v_{obs} and embeds it in a control packet sent to the switch. The switch performs an atomic conditional reset [33]: it sets the counter to zero only if its current value still equals v_{obs} . If a new read has incremented the counter between detection and reset, the conditional reset is skipped, and the controller re-evaluates the counter in the next monitoring round.

The switch-to-server ACK loss is particularly critical: the server retransmits its response upon a timeout, causing the switch to apply a duplicate lock decrement, violating system correctness. Fletch prevents this via a per-server sequence-number protocol. Each server tags every lock-related response with a monotonically increasing sequence number (initialized to 0) and increments it only upon receiving an ACK from the switch. The switch maintains a per-server expected sequence number in a sequence counter array. Upon receiving a response, the switch compares its embedded sequence number against the expected value. If they match, the switch processes the lock update, forwards the response to the client, returns an ACK to the server, and increments the expected value. If the embedded sequence number is strictly less than the value, the response is a retransmission; the switch returns an ACK to suppress further retransmissions without applying any lock update.

Server failure. Server failures are handled by the backend distributed file system, which provides the necessary replication and fault tolerance to preserve file-system consistency independently of Fletch. As Fletch employs write-through caching, every cached metadata entry is always consistent with the corresponding server-side record, and no additional cache reconciliation is required upon server recovery.

Controller failure. A controller failure does not affect requests currently in flight, as the switch data plane and backend servers operate independently during request serving. The controller’s role in the request path is limited to cache admission and eviction, both of which are non-critical for serving existing cache entries. Any ongoing cache admission is gracefully aborted via the server-side timeout mechanism described in §VII-A, and Fletch continues to serve cached requests from

the switch until the controller recovers (§VII-C).

Client failure. A client crash does not affect the correctness of the in-switch cache or the server-side namespace. Clients hold only local path-token pairs (transient state used for hash-collision resolution) and carry no authoritative cache records. Upon recovery, the client’s path-token map is empty. Each subsequent request experiences at most one cache miss per cached path to retrieve the correct token from the server (§VI-A), after which normal cache-hit operation resumes.

Network partition. A partition between the controller and the switch halts new cache admissions and evictions, but requests for already-cached entries are still served by the switch data plane and backend servers. During the partition, the cache content remains unchanged because no admission or eviction can proceed. Once the partition heals, the controller cross-checks its cached-path records against the switch’s current register state and reissues any pending admission or eviction operations before resuming normal operations.

C. System Recovery

When the switch, controller, or server experiences a total crash, Fletch restores a globally consistent state through a targeted recovery procedure that minimizes downtime and avoids a full cache cold start whenever possible.

Controller failure. If only the controller fails, the switch continues serving cached requests normally during the outage. Upon controller restart, the controller reconstructs its path-token and hash-token maps from the historical log in its local persistent key-value store (§VI-A), which records all token assignments, including those for previously evicted paths. This ensures that re-admitted paths retain their original tokens, preserving consistency with any path-token pairs cached at clients. Since all token assignments are recovered from persistent storage, the controller restores a complete view of the cached namespace without flushing the switch cache or re-running hot-path identification, and then resumes admission and eviction responsibilities with minimal disruption.

Switch failure. A switch failure resets all in-switch data-plane state, including the metadata cache, hash-token map, lock counters, and validation flags. Upon reboot, Fletch replays cache admission for each path recorded in the controller’s active log (§VI-A): the controller re-fetches the corresponding metadata from the backend servers and re-installs both the metadata and the hash-token map into the switch data plane, with each path retaining its original token. This bypasses the initial hot-path identification phase and enables rapid cache warm-up, so Fletch can quickly restore near-peak performance.

Server failure. Server failures are recovered by the backend distributed file system. Since Fletch uses write-through caching, the cached metadata in the switch remains consistent with the server’s last committed state and does not need to be reconciled upon server recovery. However, the server’s path-token map (§VI-A) is in server memory and is lost upon a failure. To reconstruct it, the recovered server queries the controller, which extracts the path-token entries assigned to this server from the active log (§VI-A) and sends them back via UDP with a timeout-and-retry mechanism for reliable delivery. Once restored, the

server resumes serving requests, and any cache entries assigned to the recovered server remain valid.

VIII. IMPLEMENTATION

We have implemented a prototype of Fletch on the Tofino switch chipset [10], comprising the controller, in-switch cache, servers, and clients. Tofino switches build on the Protocol-Independent Switch Architecture (PISA) under the RMT paradigm [17] and are programmed in P4 [16]. Although our prototype targets Tofino switches, the core techniques (i.e., path-aware cache management (§IV), multi-level read-write locking (§V), and local hash collision resolution (§VI)) can be ported to any other RMT-based programmable ASIC [1], [7], [22] that supports packet recirculation and register arrays.

Controller. The controller is implemented in C++ with 2.8 K LoC. It leverages APIs provided by the Tofino switch compiler [10] to interact with the switch data plane, including configurations and updates of MATs and registers, for cache admission and eviction (§IV) and token management (§VI). It uses RocksDB [12] (v6.22.1) to persist the active and historical logs for failure recovery.

In-switch cache. The in-switch cache is implemented in P4 with 5.1 K LoC and compiled for a Tofino switch [9], [10]. It comprises several components: (i) a hash-token map using MATs for cache lookups and local hash collision resolution, where each entry is a 9-byte key (8-byte hash key and 1-byte token); (ii) 32 register arrays of 32-bit slots for storing cache entries; (iii) a three-row CMS with three register arrays, each with 64K 16-bit slots; (iv) a frequency counter array with a register array of 32-bit slots; (v) lock counter arrays with eight register arrays, each with 64K 16-bit slots; (vi) a validation array with a register array of 1-bit slots; and (vii) a sequence counter array with a register array of 8-bit slots.

Server implementation. Fletch builds on Hadoop Distributed File System (HDFS) [53], a popularly deployed distributed file system, where a *namenode* (i.e., metadata server) manages the namespace and metadata. Each server is implemented in C++ with 3 K LoC and hosts an HDFS (v3.2) namenode [5]. We use RBF [6] with the HASH_ALL policy for namespace and metadata management, which uses consistent hashing to distribute files evenly across all namenodes for load balancing and creates directories on all namenodes. Each server manages a subset of the metadata namespace, connects to its local namenode via the C++ HDFS client library `libhdfs3` [3], and serves client requests while maintaining a path-token map for local hash resolution (§VI).

Client implementation. The C++ client driver supports multi-threaded workload execution, where each thread simulates a logical client. Each client maintains a path-token map for local hash collision resolution (§VI). It communicates with the switch via UDP-encapsulated packets customized for metadata requests, with re-transmission support for reliability. We integrate the driver with the `mdtest` [11] benchmarking tool for HDFS metadata operations.

Scope of implementation. Our prototype covers all components described in §IV–§VI. Table I summarizes the implementation status of the fault-tolerance mechanisms in §VII. The

Table I: Implementation status of failure-handling mechanisms.

Mechanism (Section)	Status
Packet loss handling during admission (§VII-A)	✓
Server-side admission-stall timeout (§VII-A)	✗
Sequence-number protocol for switch-to-server ACK loss (§VII-B)	✓
Stale lock counter monitoring (§VII-B)	✗
Network partition reconciliation (§VII-B)	✗
Controller/switch/server recovery (§VII-C)	✓

unimplemented mechanisms are orthogonal to the core caching framework and can be added without altering Fletch’s design. **Portability beyond programmable switches.** Fletch is prototyped on a Tofino P4-PISA switch, yet its core techniques are not inherently tied to the Tofino architecture and are portable to other programmable hardware platforms. One possible future direction is a Data Processing Unit (DPU), which sits at the host edge between the NIC and the host CPU. In a DPU-based deployment, Fletch would transition from a network-core cache to a host-edge accelerator, eliminating the kernel file-system stack traversal and RPC round-trip overhead for metadata requests, since the DPU intercepts them before they reach the host. A complete mapping of Fletch’s core mechanisms (e.g., path resolution, lock counter placement, and multi-DPU coordination for cache consistency) onto DPU hardware requires a non-trivial design and is beyond the scope of this paper.

IX. EVALUATION

A. Methodology

Testbed. We evaluate Fletch on a testbed comprising a 3.2 Tbps two-pipeline Tofino switch [9], [10] and three physical machines. Two machines host servers, each with a 2.40 GHz 10-core Intel Xeon Silver 4210R CPU, 128 GiB DRAM, and a 2 TB HDD (Dell PERC H330 Mini). The client driver runs on the remaining machine, with a 2.40 GHz, 16-core Intel Xeon Silver 4314 CPU, 128 GiB DRAM, and a 960 GB NVMe SSD (Micron 9300 PRO). Each machine is connected to the switch via a 40 Gbps NIC (Mellanox ConnectX-5 CX516A). The client machine uses one pipeline, and the two server machines use another pipeline. All counter arrays (§VIII) reside in the server-connected pipeline.

Since our two-pipeline Tofino switch lacks native support for cross-pipeline recirculation, we physically connect the designated ingress pipeline hosting the lock counter arrays to another ingress pipeline using a physical wire [52]. This enables requests from another ingress pipeline to be recirculated to the designated ingress pipeline (§V). As a result, every client request incurs one cross-pipeline recirculation to access the lock counter arrays in the server-connected pipeline. Future programmable switches with native cross-pipeline recirculation would eliminate this requirement.

Workloads. We evaluate Fletch using four real-world workload traces: (i) Alibaba’s Pangu file-system instances (Alibaba) [44], (ii) convolutional neural network training (Training) [62], (iii) the processing of one million thumbnail images (Thumb) [62], and (iv) a LinkedIn HDFS cluster (LinkedIn) [49]. Table II

Table II: Real-world workloads and their metadata operation ratios.

Workload	Operation ratio	Read ratio
Alibaba [44]	52.6% open/close, 9.59% create, 3.9% readdir, 0.1% chmod, 11.9% delete, 12.4% stat, 0.2% statdir, 0.005% mkdir, 0.005% rmdir, 9.3% file rename	69.1%
Training [62]	54.3% open/close, 27.16% stat, 0.13% readdir, 9.01% create, 0.13% mkdir, 0.13% rmdir, 9.01% delete, 0.13% statdir	81.7%
Thumb [62]	57.01% open/close, 28.44% stat, 0.13% readdir, 14.16% create, 0.13% mkdir, 0.13% statdir	85.7%
LinkedIn [49]	84% open/getattr, 9% create/mkdir, 7% chmod/delete/rename	84%

summarizes the proportion of metadata operations and the read ratio for each workload.

We refine the workloads for our evaluation. To focus on metadata performance, we exclude reads and writes of file data, following prior studies [39], [44], [46], [49]. For Training and Thumb that include file reads and writes [62], we exclude these operations and normalize the remaining metadata operations. Since we exclude file writes, we treat `close` as a read operation, while it updates both modification and access timestamps if the closed file has been updated. HDFS updates access timestamps hourly by default [4], and we exclude timestamp updates as they are infrequent.

For LinkedIn, as the original paper [49] does not provide operation ratios, we derive them from trace analysis [44]: `open` (42%), `getattr` (42%), `create` (4.5%), `mkdir` (4.5%), `chmod` (1%), `delete` (3%), and `rename` (3%). We assign the smallest ratio to `chmod`, which is less frequent than `delete` and `rename` [44]. We replace `getattr` (which corresponds to `stat` and `statdir` in HDFS) with `stat`, as file operations are much more frequent than directory operations [44], and focus on file renaming for `rename`.

Generation of metadata operations. We use `mdtest` [11] to generate file-system namespaces and metadata operations. We configure a default path depth of nine, as metadata requests are often aggregated at small depths [14], [25], [45]. We create 32 million empty files to focus on metadata performance, following prior evaluation [39], [44], [46], [49], [58], [61]. To simulate workload skewness, we apply the 80/20 rule (80% of operations on 20% of directories and files [57], [62]) and model file access frequencies across levels using a power-law distribution with an exponent of 0.9, with operation types assigned randomly using the ratios in Table II. For `statdir` and `readdir`, we select the parent directory of the chosen file; for `mkdir` and `rmdir`, we use separate directories to avoid removing non-empty directories. We issue 32 million metadata requests per workload and address various workload settings in §IX-C.

We mix all metadata operations in each workload. However, `rename`, `delete`, and `rmdir` involve metadata modifications that necessitate the granting and revoking of leases in HDFS [55], and the lease-based operations on frequently created and deleted files can slow down all metadata operations under stress tests. Thus, we place them at the end of the request sequence to keep metadata operations at high rate.

Scaling servers. Although our testbed contains only two physical servers, our Tofino switch has significantly higher forwarding throughput (3.2 Tbps [9]) than server throughput

(tens of KOPS) and is not our evaluation bottleneck. To simulate larger-scale deployment with limited hardware while maintaining realism in evaluation, we adopt the well-known *server rotation* approach as in prior in-switch caching studies [32], [52]. Let S be the number of simulated servers. We assign files to these S servers using HDFS’s RBF policy (§VIII). We calculate each server’s load from file access frequencies and identify the bottlenecked server with the highest load. We then perform our evaluation in S iterations. We first deploy the bottlenecked server in one physical machine and fully saturate it to measure its performance. In the following $S - 1$ iterations, we reset the physical machines to the initial state, pair the bottlenecked server with one of the $S - 1$ non-bottlenecked servers, deploy them on the two physical machines, and measure the performance of the non-bottlenecked server. We aggregate the performance of all servers as the overall performance. By default, we simulate 16 servers and increase the scale to 128 servers (Exp#1).

Baselines. We consider *client-side caching* (CCache) by faithfully following the state-of-the-art client-side caching implementations in IndexFS [49] and InfiniFS [44] (§X). CCache uses RocksDB (v6.22.1) as a key-value store to keep all metadata instead of in an HDFS namenode in each simulated server to eliminate HDFS’s path resolution overhead. Each CCache client locally caches directories’ permission metadata, and forwards all read requests to servers for attribute retrieval. We implement lazy invalidation [44] for cache consistency, which outperforms lease-based cache management [49]. We do not compare Fletch with in-switch key-value cache systems (e.g., NetCache [32] and FarReach [52]) since they are designed for key-value stores and cannot ensure correctness or consistency for file-system operations.

We evaluate four schemes organized by deployment mode (§III-C). In the standalone mode: (i) *NoCache*, which performs metadata operations directly with HDFS namenodes without caching; and (ii) *Fletch*, which extends NoCache with in-switch caching. In the complementary mode: (iii) *CCache*, our client-side caching implementation; and (iv) *Fletch+*, which extends CCache with in-switch caching.

Before each experiment, we pre-load 32 million files into HDFS namenodes for NoCache and Fletch, and the corresponding paths and metadata into RocksDB for CCache and Fletch+. We pre-load the 5,000 hottest files and their ancestors into the in-switch cache for Fletch and Fletch+. We set the pre-defined CMS threshold of Fletch and Fletch+ to 10 and 20, respectively (a larger threshold is used for Fletch+ due to its higher throughput with client-side caching), and reset the CMS and the frequency counter array every two seconds. We simulate 128 client threads, which sufficiently saturate backend servers. The client-side cache of each simulated client in CCache and Fletch+ is allocated 4 MiB [44]. We plot the average results over five runs, with error bars as 95% confidence intervals under the Student’s t -distribution.

B. Performance Analysis

(Exp#1) Throughput under real-world workloads. Figure 7 shows throughput on 16 and 128 simulated servers under four

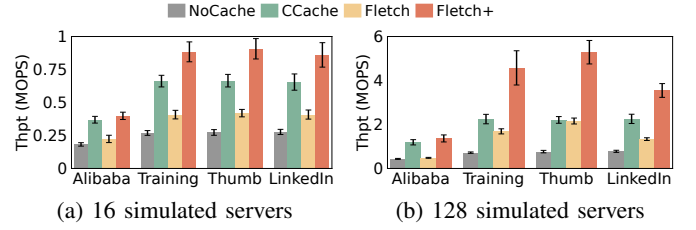


Figure 7: (Exp#1) Performance under real-world workloads.

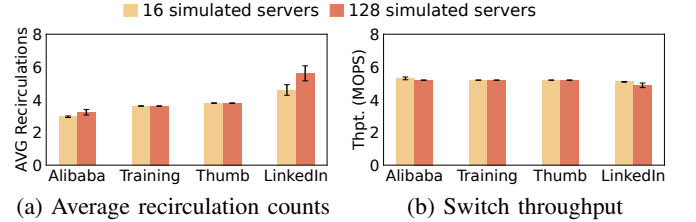


Figure 8: (Exp#1) Recirculation overhead of Fletch on real workloads.

real-world workloads. Under 16 servers, in the standalone mode, Fletch increases NoCache’s throughput by 22.9%, 51.7%, 53.7%, and 47.6% in Alibaba, Training, Thumb, and LinkedIn, respectively, as in-switch caching absorbs skewed hot-path requests that otherwise cause load imbalance in NoCache. In the complementary mode, Fletch+ increases CCache’s throughput by 8.0%, 33.5%, 36.3%, and 31.5% in the same workloads, respectively, as CCache forwards all attribute-retrieval requests to servers and cannot resolve server-side load imbalance, while Fletch+ adds an in-switch caching layer that absorbs these requests before they reach the bottlenecked server.

Fletch has lower throughput than CCache by around 38.1% across all workloads, as CCache reduces path resolution overhead by caching directory permission metadata at clients and bypasses HDFS lease management and distributed transaction overhead. This is expected: client-side caching and in-switch load balancing are orthogonal optimizations targeting distinct bottlenecks, which is precisely why the complementary mode combines both.

Under 128 servers, the gains in both modes increase substantially. In the standalone mode, Fletch increases NoCache’s throughput by 11.0%, 134.6%, 181.6%, and 71.2% in Alibaba, Training, Thumb, and LinkedIn, respectively. In the complementary mode, Fletch+ increases CCache’s throughput by 14.7%, 103.6%, 139.6%, and 57.3% for the same workloads, respectively. The gains are significantly higher than in 16 servers (except in Alibaba) as they improve server scalability via load balancing. The exception in Alibaba is attributed to its high write ratio: under write-through caching, cache consistency maintenance for frequent writes incurs substantial overhead that limits scalability gains. Across both scales, the results confirm that the two modes target distinct deployment regimes: the standalone mode offers sizable improvements over a no-cache deployment, while the complementary mode offers further gains on top of an existing client-side caching deployment.

We further analyze the recirculation overhead of Fletch on 16 and 128 simulated servers. We measure the *recirculation count* (i.e., the average number of recirculations per request) for each workload using the Barefoot Shell monitoring tool

[2]. Following the server rotation methodology (§IX-A), in each iteration, a physical machine acts as one simulated server and processes its assigned share of the workload’s 32 million requests; we read the number of packets passing through the switch’s recirculation port at the end of each iteration. Summing these counts across all iterations and dividing by 32 million yields the recirculation count.

Figure 8(a) shows the recirculation count results. Across all workloads, the count ranges from 3.00 to 5.61, and includes two components: (i) one mandatory cross-pipeline recirculation per request since the client-connected pipeline and the server-connected pipeline hosting the lock counter arrays are physically separate in our testbed (§IX-A); and (ii) the remaining recirculations from in-switch path resolution (§IV-A) for reads and lock acquisition (§V-B) for writes. The counts remain modest across all workloads, showing that Fletch’s recirculation overhead is manageable under realistic access patterns.

We next evaluate the switch’s processing capacity under the measured recirculation counts to quantify the available switch-side headroom. To isolate switch throughput from backend server performance, we pre-load the 5,000 hottest files and their ancestors into the in-switch cache (§IX-A) and use a DPDK-based client [8] to issue `stat` requests exclusively on these cached paths, so that all requests are served entirely by the switch on cache hits without any server involvement.

A cache-hit `stat` request on a path at depth L always incurs exactly $L + 2$ recirculations: L for in-switch path resolution (one recirculation per level, for levels 1 through L ; §IV-A), one for decrementing the lock counter at the final level after path resolution completes, and one cross-pipeline recirculation in our testbed (§IX-A). To reproduce a target recirculation count r from Figure 8(a), we randomly select a cached path at depth $\lceil r \rceil - 2$, rounding up r to the nearest integer to ensure a conservative evaluation since path depth must be an integer. Under this setup, we increase the client’s sending rate until the switch begins dropping packets internally (i.e., the client receives fewer responses than requests sent while its receive buffer does not overflow) and report the highest sustained throughput before this saturation point.

Figure 8(b) shows the switch’s peak throughput under the recirculation counts of the four workloads. Fletch achieves 5.1–5.3 MOPS and 4.9–5.2 MOPS on 16 and 128 servers. The switch peak throughput decreases as the recirculation count increases, indicating that recirculation is a switch-side overhead of Fletch’s design. Nevertheless, within the recirculation regime observed under real-world workloads (3.00–5.61 from Figure 8(a)), the switch sustains a throughput at least $6.3\times$ and $2.2\times$ higher than the aggregate throughput of NoCache and CCache on 128 servers (Figure 7(b)), respectively. This confirms that, under the recirculation counts induced by the evaluated workloads, Fletch and Fletch+ achieve substantial throughput gains over their respective baselines. Note that the switch throughput does not exceed the aggregate throughput of Fletch+ on 128 servers. This exposes a limitation of server rotation: when a system’s aggregate throughput approaches the switch’s processing capacity, server rotation may overestimate the actual achievable throughput. In Appendix, we evaluate the recirculation overhead under high write ratios.

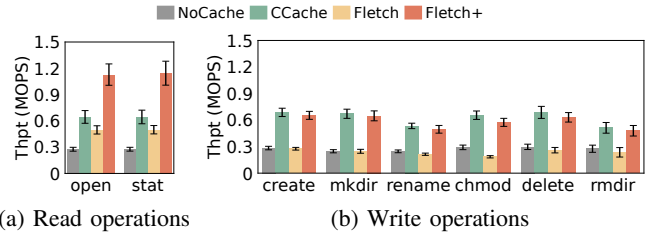


Figure 9: (Exp#2) Single-operation performance.

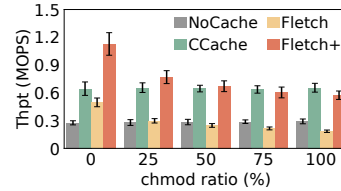


Figure 10: (Exp#3) Throughput versus `chmod` ratio.

Table III: (Exp#3) Recirculation count versus `chmod` ratio.

chmod ratio	SingleLock	MultiLock
0%	7.63	7.63
25%	442.06	37.21
50%	436.29	39.42
75%	369.43	25.67
100%	1	1

(Exp#2) Single-operation performance. Figure 9 shows the throughput of individual metadata operations. For read operations (`open` and `stat`), Fletch increases NoCache’s throughput by 80.1% and 80.5%, while Fletch+ increases CCache’s throughput by 74.9% and 77.5%, respectively. For write operations (`create`, `mkdir`, `rename`, `chmod`, `delete`, and `rmdir`), Fletch has lower throughput than NoCache by 2.7%, 0.2%, 13.2%, 36.5%, 12.7%, and 14.6%, respectively, while Fletch+ has lower throughput than CCache by 4.9%, 3.5%, 7.2%, 12.3%, 8.1%, and 6.4%, respectively. The performance drops stem from the switch’s cache maintenance overhead. Among write operations, `chmod` incurs the highest overhead as `chmod` on cached paths requires fetching metadata from HDFS namenodes and updating the in-switch cache; in contrast, `rename`, `delete`, and `rmdir` on cached paths only mark paths as deleted, and `create` and `mkdir` only involve creations on new files and directories on HDFS namenodes, respectively, and will not trigger cache updates. Although writes incur overhead due to consistency maintenance, writes are rare in realistic workloads, and both Fletch and Fletch+ maintain their performance gains (Exp#1).

(Exp#3) Impact of `chmod` ratio. We further analyze `chmod`, which triggers frequent cache updates and shows the most performance drops in Fletch and Fletch+ (Exp#2). We generate mixed read-write workloads composed of `open` (i.e., reads) and `chmod` (i.e., writes) with different ratios. Each of `open` and `chmod` follows a power-law distribution with an exponent of 0.9. Figure 10 shows that at 0% `chmod` ratio, Fletch and Fletch+ achieve higher throughput than NoCache and CCache, respectively, but as the `chmod` ratio increases, their throughput decreases, while the throughput of NoCache and CCache remains stable. Fletch and Fletch+ begin to show throughput degradations when the `chmod` ratio exceeds 50%. At 100% `chmod` ratio, Fletch and Fletch+ reach the lowest throughput, 36.5% and 12.3% lower than NoCache and CCache, respectively. Nevertheless, real-world workloads have low `chmod` ratios (Table II), so Fletch and Fletch+ still maintain performance gains in practice (Exp#1).

Fletch’s multi-level read-write locking design (§V) signifi-

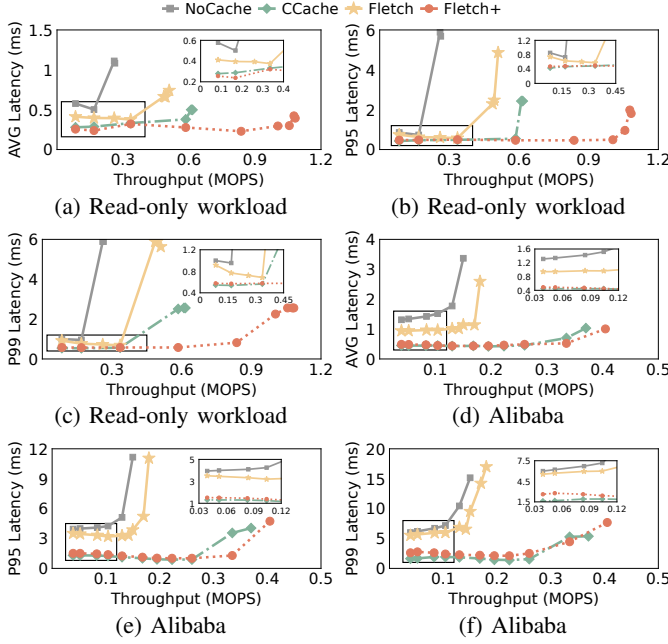


Figure 11: (Exp#4) Latency analysis.

cantly reduces the overhead in writes. We compare multi-level locking (*MultiLock*) with single-level locking (*SingleLock*), which always maps a full path to the first lock counter array. We measure the recirculation count as described in Exp#1. Table III shows that for read-only and write-only workloads, the recirculation counts for both *SingleLock* and *MultiLock* are 7.63 and 1, respectively, as *Fletch* recirculates a read request multiple times for both path resolution and lock counter access, while a write request requires only one recirculation for lock counter access. *MultiLock* significantly reduces the recirculation count of *SingleLock* (e.g., by 93.1% for 75% *chmod*) by mitigating lock contention.

(Exp#4) Latency analysis. We analyze request latencies by adjusting the request sending rate to a target aggregate throughput, as in prior studies [21], [24], [32], [52], so as to analyze the trade-off between latencies and throughput. We focus on (i) a read-only workload that issues 32 million open requests and (ii) the Alibaba workload, which has the largest write ratio (Table II). The two workloads show *Fletch*’s best- and worst-case performance, respectively, under write-through caching (§III-A). We follow a power-law distribution with an exponent of 0.9 and consider 16 simulated servers.

Figure 11 shows the average, p95, and p99 latency results. For the read-only workload, all schemes show low latencies at low target throughput (less than 0.1 MOPS) as the servers are not saturated and do not have queueing delays. When the target throughput increases to 0.2 MOPS, *NoCache* starts to show increasing latencies due to load imbalance under higher loads. *Fletch* maintains low latencies via in-switch caching. For example, at target throughput 0.26 MOPS, *Fletch* reduces *NoCache*’s average, p95, and p99 latencies by 64.6%, 89.8%, and 87.7%, respectively. The same trend is also observed for *Fletch+* and *CCache*. For example, at target throughput 0.59 MOPS, *Fletch+* reduces *CCache*’s average, p95, and p99

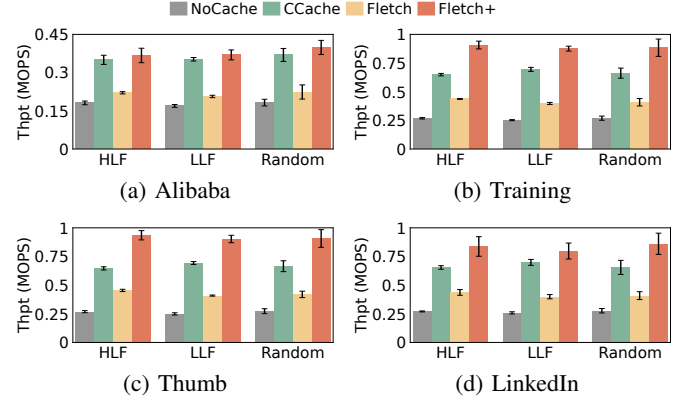


Figure 12: (Exp#5) Impact of file access frequency assignment.

latencies by 26.9%, 14.7%, and 77.0%, respectively.

For the Alibaba workload, *Fletch* also outperforms *NoCache*. For example, at target throughput 0.15 MOPS, *Fletch* reduces *NoCache*’s average, p95, and p99 latencies by 66.1%, 65.4%, and 37.2%, respectively. For *Fletch+* and *CCache*, at low throughput, they have comparable average and p95 latencies, while *Fletch+* has slightly higher p99 latencies due to the switch’s cache maintenance overhead. At high throughput (e.g., 0.35 MOPS), *Fletch+* reduces *CCache*’s average, p95, and p99 latencies by 25.5%, 63.7%, and 15.5%, respectively.

C. Impact of Workload Settings

(Exp#5) Impact of file access frequency assignment. We generate a sequence of access frequencies in descending order based on a power-law distribution with an exponent of 0.9, and another sequence of files based on different sorting orders. We assign the i -th access frequency to the i -th file. We consider three sorted file sequences: (i) *high-level-first (HLF)*, which sorts files in descending order of their levels (i.e., files at higher levels have higher access frequencies), (ii) *low-level-first (LLF)*, which sorts files in ascending order of their levels (i.e., files at lower levels have higher access frequencies), and (iii) *random* (our default), which generates a random sequence of files across all levels. Figure 12 shows that *Fletch* and *Fletch+* still outperform *NoCache* and *CCache*, respectively, under different file access frequency assignments. For example, in Thumb (the most read-intensive), *Fletch* and *Fletch+* increase the throughput of *NoCache* and *CCache* by 53.7-69.0% and 30.2-44.7%, respectively.

(Exp#6) Impact of access skewness. We vary the skewness level for access frequencies under the uniform distribution and varying power-law distributions with an exponent of 0.8, 0.9 (our default), and 1.0. A larger exponent implies a more skewed access pattern. Figure 13 shows that under the uniform workload, *Fletch* and *Fletch+* have slightly less throughput than *NoCache* and *CCache*, respectively, by up to 5.0%, since most requests come from uncached paths and are served by servers, while *Fletch* and *Fletch+* incur extra overhead for maintaining cache consistency.

For Training and Thumb, for more skewed access, *Fletch* and *Fletch+* show increasing throughput via in-switch caching, while *NoCache* and *CCache* show decreasing throughput due

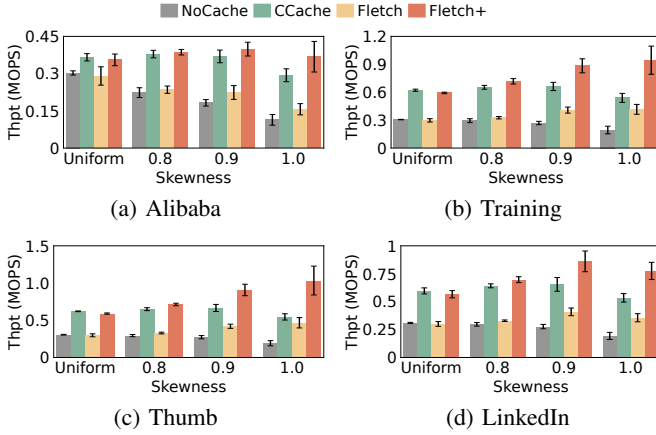


Figure 13: (Exp#6) Impact of access skewness.

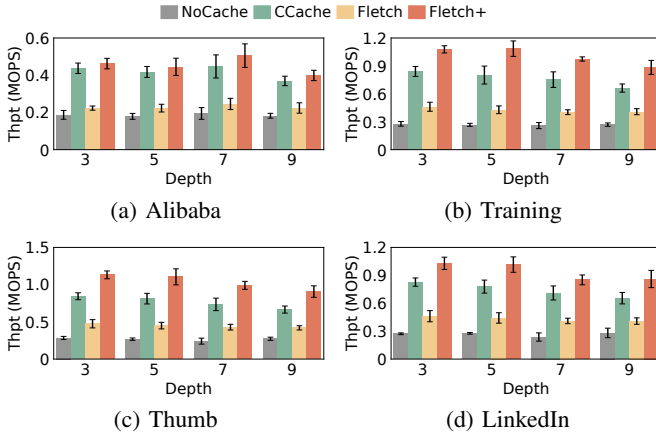


Figure 14: (Exp#7) Impact of maximum path depth.

to more severe load imbalance. For example, for Thumb at exponent 1.0, Fletch and Fletch+ achieve throughput gains of $2.4\times$ and $1.9\times$ over NoCache and CCache, respectively.

For Alibaba and LinkedIn, as the exponent increases from 0.9 to 1.0, the throughput of Fletch and Fletch+ decreases. Alibaba has the largest write ratio, while LinkedIn has the highest `chmod` ratio. They incur significant maintenance overhead for cache consistency. Nevertheless, at exponent 1.0 for LinkedIn, Fletch and Fletch+ still increase the throughput of NoCache and CCache by 84.3% and 45.1%, respectively.

(Exp#7) Impact of maximum path depth. We vary the maximum path depth as 3, 5, 7, and 9 (our default). Figure 14 shows that Fletch and Fletch+ always outperform NoCache and CCache, respectively. For example, in Thumb, Fletch and Fletch+ increase the throughput of NoCache and CCache by 53.7-77.9% and 34.1-36.3%, respectively.

(Exp#8) Impact of dynamic workloads. We consider dynamic workloads with varying access frequencies of files over time. We follow the prior studies [32], [40], [52] to generate the *hot-in* dynamic pattern, which periodically selects the 100 least-frequently accessed files, re-assigns them with the highest access frequencies, and adjusts the access frequencies of other files accordingly to maintain a power-law distribution. We set the change period as 20 seconds and run each scheme for 200 seconds to measure per-second throughput. We disable server rotation as the system states change under dynamic

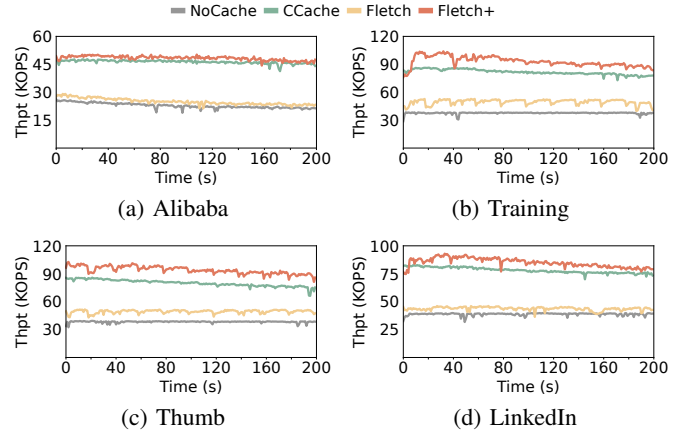


Figure 15: (Exp#8) Impact of dynamic workloads.

workloads; instead, we issue workloads to the two physical servers and measure performance directly.

Figure 15 shows that for Training, Thumb, and LinkedIn, Fletch and Fletch+ show performance dips due to periodic changes of file access frequencies. Before new hot records are admitted, performance dips occur, but Fletch and Fletch+ quickly admit new hot records and return to high performance with path-aware cache management (§IV). Also, local hash collision resolution (§VI) incurs minimal overhead to cache admission and eviction. For Alibaba, which has the largest write ratio, Fletch and Fletch+ have marginal performance gains over NoCache and CCache, respectively, as they incur cache consistency overhead. Nevertheless, Fletch and Fletch+ still effectively respond to dynamic workloads.

D. Switch Deployment

(Exp#9) Switch resource usage. We measure the resource usage on the Tofino switch for (i) SRAM (15 MiB), (ii) number of stages (12), (iii) number of ALUs (48), and (iv) PHV size (768 bytes). We also quote the resource usage of two in-switch key-value cache systems, NetCache [32] and FarReach [52], based on the switch resource usage reported in the paper [52]. Table IV shows that NoCache and CCache use the least resources for L2/L3 forwarding, while Fletch and Fletch+ also support L2/L3 forwarding to process packets unrelated to metadata operations at line rate (3.2 Tbps for the Tofino switch [9]). Additionally, Fletch and Fletch+ consume additional resources for caching (e.g., using SRAM and ALUs to cache metadata, track access frequencies, and maintain lock counters, and using PHVs to parse metadata requests), yet their resource usage is comparable to NetCache and FarReach (state-of-the-art in-switch key-value cache approaches).

Fletch's resource footprint is configurable through lock counter array sharing across path levels. In our current prototype, we already apply this by having path levels 8 and beyond share a single lock counter array, keyed by the level-8 hash component, to reduce stage usage (§V-A). This principle generalizes to shallower levels: adjacent levels can be merged under a single shared array. For example, levels 6, 7, and 8 can share level 6's lock counter array as follows. For a read request at depth 8, the switch increments the counter indexed

Table IV: (Exp#9) Switch resource usage (numbers in brackets refer to the fractions of used resources over total available ones). Numbers of NetCache and FarReach are quoted from [52].

Scheme	SRAM (KiB)	# Stages	# ALUs	PHV size (bytes)
NoCache	288 (1.9%)	4 (33.3%)	0 (0%)	256 (33.3%)
CCache	288 (1.9%)	4 (33.3%)	0 (0%)	256 (33.3%)
NetCache [32]	7856 (51.2%)	12 (100%)	45 (93.8%)	528 (68.8%)
FarReach [52]	8080 (52.6%)	12 (100%)	45 (93.8%)	499 (65.0%)
Fletch	8976 (58.4%)	12 (100%)	47 (97.6%)	712 (92.7%)
Fletch+	8976 (58.4%)	12 (100%)	47 (97.6%)	712 (92.7%)

by the path’s level-6 hash key once; during path resolution, it does not decrement this counter when processing levels 6 or 7, and decrements it only once after resolving level 8. For a write request at depth 8 awaiting a lock, the switch checks the same counter indexed by the level-6 hash key. Under this scheme, levels 7 and 8 no longer require dedicated arrays, freeing up pipeline stages and ALUs for other on-switch applications. The sharing configuration is specified at compile time. The trade-off is that the shared counter aggregates active read counts across all covered levels, thereby increasing lock acquisition latency relative to dedicated per-level arrays. In exchange, Fletch retains its full cache depth and hit ratio even with fewer pipeline stages and ALUs, providing a tunable trade-off between write lock acquisition latency and hardware resource efficiency.

E. System Recovery

In Appendix, we evaluate crash recovery time under switch, controller, and server failures. We find that switch recovery takes 14.6–75.5 s for 5 K–20 K paths due to sequential cache-admission replay, while controller and server recovery are substantially faster (9.9–38.8 ms and 0.5–2.1 s, respectively).

X. RELATED WORK

Scaling file-system metadata management. Metadata partitioning distributes file-system management across multiple servers for scalability. There are two primary approaches: (i) *dynamic sub-tree partitioning*, which distributes namespace sub-trees across servers [51], [57], [59], and (ii) *hash-based partitioning*, which distributes file-system metadata across servers via hashing [41], [46]–[48], [54], [58]. Fletch complements these approaches as an in-switch cache that absorbs operations upstream of file-system metadata layers.

Client-side caching. PanFS [60] caches file and directory metadata and provides callbacks for cache consistency. IndexFS [49] and LocoFS [39] use lease-based caching for metadata management and invalidate cache entries upon lease expiration, but incur high overhead for renewing cache entries’ leases. InfiniFS [44] applies lazy invalidation for directory access metadata to limit the overhead of lease-based caching. Client-side caching often incurs high client-side complexity and overhead in maintaining cache consistency across a large number of clients. Fletch simplifies cache consistency by caching file-system metadata in a programmable switch that lies on the critical paths of multiple clients.

In-switch caching. Programmable switches have been extensively studied for concurrency control [37], [66], network

monitoring [27], replication coordination [31], remote procedure calls [65], key-value stream aggregation [30], and distributed lock management [63], [64]. Several studies explore in-switch write-through caching: SwitchKV [40] and NetCache [32] target read-intensive workloads; DistCache [43] supports caching across multiple switches; OrbitCache [34] supports variable-length items. Pegasus [38] and TurboKV [26] cache replica-to-server mappings for replica selection. Mind [35] caches object-to-memory mappings for disaggregated memory systems. Concordia [56] tracks cache copy locations and states to address concurrency in shared memory systems. ICH [20] and HarmonyCache [19] design write-back caching for dynamic workloads and multi-switch read-write separation, respectively. FarReach [52] designs fault-tolerant write-back caching for write-intensive workloads, and its extension, DistReach [52], supports multiple switches. The above in-switch caches target key-value stores, which differ from file-system semantics. SwitchFS [62] proposes in-switch tracking of directory updates, while maintaining a client-side metadata cache for path resolution. In contrast, Fletch moves file-system metadata caching to switches, and also complements client-side caching.

XI. CONCLUSION

Fletch is an in-switch file-system metadata caching framework, aiming to achieve high throughput and load balancing for distributed file-system metadata services. It employs path-aware cache management, multi-level read-write locking, and local hash collision resolution. Experiments on a Tofino-switch testbed show that Fletch achieves significant throughput gains and complements client-side caching.

REFERENCES

- [1] “Broadcom Trident 5 programmable Ethernet switch series,” <https://www.broadcom.com/products/ethernet-connectivity/switching/strataxgs/bcm78800>.
- [2] “Cisco Barefoot Shell,” https://www.cisco.com/c/en/us/td/docs/switches/datacenter/nexus9000/sw/92x/programmability/guide/b-cisco-nexus-9000-series-nx-os-programmability-guide-92x/b-cisco-nexus-9000-series-nx-os-programmability-guide-92x_chapter_0110.html.
- [3] “HDFS C/C++ Library,” <https://github.com/erikmutterbach/libhdfs3>.
- [4] “HDFS default configurations in Hadoop 3.2.4,” <https://hadoop.apache.org/docs/r3.2.4/hadoop-project-dist/hadoop-hdfs/hdfs-default.xml>.
- [5] “HDFS in Hadoop 3.2.4,” <https://hadoop.apache.org/docs/r3.2.4/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>.
- [6] “HDFS Router-based Federation in Hadoop 3.2.4,” <https://hadoop.apache.org/docs/r3.2.4/hadoop-project-dist/hadoop-hdfs-rbf/HDFSRouterFederation.html>.
- [7] “Huawei CloudEngine series data center switches,” <https://carrier.huawei.com/en/products/fixed-network/b2b/ethernet-switches/dc-switches#myCarousel2>.
- [8] “Intel Data Plane Development Kit (DPDK),” <https://www.dpdk.org/>.
- [9] “Intel Tofino 3.2 Tbps, 2 pipelines,” <https://www.intel.com/content/www/us/en/products/sku/218641/intel-tofino-3-2-tbps-2-pipelines/specifications.html>.
- [10] “Intel Tofino native architecture,” <https://github.com/barefootnetworks/Open-Tofino>.
- [11] “Mdttest HPC Benchmark,” <https://sourceforge.net/projects/mdttest/>.
- [12] “RocksDB,” <https://github.com/facebook/rocksdb/>.
- [13] C. L. Abad, N. Roberts, Y. Lu, and R. H. Campbell, “A storage-centric analysis of MapReduce workloads: File popularity, temporal locality and arrival patterns,” in *Proc. of IEEE IISWC*, 2012.
- [14] N. Agrawal, W. J. Bolosky, J. R. Douceur, and J. R. Lorch, “A five-year study of file-system metadata,” *ACM Trans. on Storage*, vol. 3, no. 3, pp. 9:1–9:32, 2007.

- [15] G. Ananthanarayanan, S. Agarwal, S. Kandula, A. Greenberg, I. Stoica, D. Harlan, and E. Harris, "Scarlett: Coping with skewed content popularity in MapReduce clusters," in *Proc. of ACM EuroSys*, 2011.
- [16] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming protocol-independent packet processors," in *Proc. of ACM SIGCOMM*, 2014.
- [17] P. Bosshart, G. Gibb, H.-S. Kim, G. Varghese, N. McKeown, M. Izzard, F. Mujica, and M. Horowitz, "Forwarding metamorphosis: Fast programmable match-action processing in hardware for SDN," in *Proc. of ACM SIGCOMM*, 2013.
- [18] P. Carns, S. Lang, R. Ross, M. Vilayannur, J. Kunkel, and T. Ludwig, "Small-file access in parallel file systems," in *Proc. of IEEE ISPD*, 2009.
- [19] J. Chen, X. Xu, and W. Wu, "HarmonyCache: Scalable in-network cache with read-write separation," *IEEE Trans. on Parallel and Distributed Systems*, vol. 37, no. 4, pp. 920–933, April 2026.
- [20] —, "ICH: In-network cache hierarchy for dynamic workloads," *IEEE Trans. on Networking*, vol. 34, no. 1, pp. 1284–1299, January 2026.
- [21] Y. Cheng, A. Gupta, and A. R. Butt, "An in-memory object caching framework with adaptive load balancing," in *Proc. of ACM EuroSys*, 2015.
- [22] S. Chole, A. Fingerhut, S. Ma, A. Sivaraman, S. Vargaftik, A. Berger, G. Mendelson, M. Alizadeh, S.-T. Chuang, I. Keslassy, A. Orda, and T. Edsall, "drmt: Disaggregated programmable switching," in *Proc. of ACM SIGCOMM*, 2017.
- [23] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.
- [24] D. Didona and W. Zwaenepoel, "Size-aware sharding for improving tail latencies in in-memory key-value stores," in *Proc. of USENIX NSDI*, 2019.
- [25] J. R. Douceur and W. J. Bolosky, "A large-scale study of file-system contents," *ACM SIGMETRICS Performance Evaluation Review*, vol. 27, no. 1, pp. 59–70, 1999.
- [26] H. Eldakiky, D. H.-C. Du, and E. Ramadan, "Scaling up the performance of distributed key-value stores with in-switch coordination," in *Proc. of IEEE MASCOTS*, 2021.
- [27] A. Gupta, R. Harrison, M. Canini, N. Feamster, J. Rexford, and W. Willinger, "Sonata: Query-driven streaming network telemetry," in *Proc. of ACM SIGCOMM*, 2018.
- [28] T. Harter, D. Borthakur, S. Dong, A. Aiyer, L. Tang, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Analysis of HDFS under HBase: A Facebook messages case study," in *Proc. of USENIX FAST*, 2014.
- [29] T. Harter, C. Dragga, M. Vaughn, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "A file is not a file: Understanding the I/O behavior of apple desktop applications," *ACM Trans. on Computer Systems*, vol. 30, no. 3, pp. 1–39, 2012.
- [30] Y. He, W. Wu, Y. Le, M. Liu, and C. Lao, "A generic service to provide in-network aggregation for key-value streams," in *Proc. of ACM ASPLOS*, 2023.
- [31] X. Jin, X. Li, H. Zhang, N. Foster, J. Lee, R. Soulé, C. Kim, and I. Stoica, "NetChain: Scale-free sub-RTT coordination," in *Proc. of USENIX NSDI*, 2018.
- [32] X. Jin, X. Li, H. Zhang, R. Soulé, J. Lee, N. Foster, C. Kim, and I. Stoica, "NetCache: Balancing key-value stores with fast in-network caching," in *Proc. of ACM SOSP*, 2017.
- [33] R. Joshi, C. H. Song, X. Z. Khooi, N. Budhdev, A. Mishra, M. C. Chan, and B. Leong, "Masking corruption packet losses in datacenter networks with link-local retransmission," in *Proc. of ACM SIGCOMM*, 2023.
- [34] G. Kim, "Pushing the limits of in-network caching for key-value stores," in *Proc. of USENIX NSDI*, 2025.
- [35] S.-s. Lee, Y. Yu, Y. Tang, A. Khandelwal, L. Zhong, and A. Bhattacharjee, "Mind: In-network memory management for disaggregated data centers," in *Proc. of ACM SOSP*, 2021.
- [36] A. W. Leung, S. Pasupathy, G. Goodson, and E. L. Miller, "Measurement and analysis of large-scale network file system workloads," in *Proc. of USENIX FAST*, 2008.
- [37] J. Li, E. Michael, and D. R. Ports, "Eris: Coordination-free consistent transactions using in-network concurrency control," in *Proc. of ACM SOSP*, 2017.
- [38] J. Li, J. Nelson, E. Michael, X. Jin, and D. R. Ports, "Pegasus: Tolerating skewed workloads in distributed storage with in-network coherence directories," in *Proc. of USENIX OSDI*, 2020.
- [39] S. Li, Y. Lu, J. Shu, Y. Hu, and T. Li, "LocoFS: A loosely-coupled metadata service for distributed file systems," in *Proc. of IEEE SC*, 2017.
- [40] X. Li, R. Sethi, M. Kaminsky, D. G. Andersen, and M. J. Freedman, "Be fast, cheap and in control with SwitchKV," in *Proc. of USENIX NSDI*, 2016.
- [41] G. Liao and D. J. Abadi, "FileScale: Fast and elastic metadata management for distributed file systems," in *Proc. of ACM SoCC*, 2023.
- [42] M. Liu, L. Luo, J. Nelson, L. Ceze, A. Krishnamurthy, and K. Atreya, "IncBricks: Toward in-network computation with an in-network cache," in *Proc. of ACM ASPLOS*, 2017.
- [43] Z. Liu, Z. Bai, Z. Liu, X. Li, C. Kim, V. Braverman, X. Jin, and I. Stoica, "DistCache: Provable load balancing for large-scale storage systems with distributed caching," in *Proc. of USENIX FAST*, 2019.
- [44] W. Lv, Y. Lu, Y. Zhang, P. Duan, and J. Shu, "InfiniFS: An efficient metadata service for large-scale distributed filesystems," in *Proc. of USENIX FAST*, 2022.
- [45] D. T. Meyer and W. J. Bolosky, "A study of practical deduplication," *ACM Trans. on Storage*, vol. 7, no. 4, pp. 1–20, 2012.
- [46] S. Niazi, M. Ismail, S. Haridi, J. Dowling, S. Grohsschmidt, and M. Ronström, "HopsFS: Scaling hierarchical file system metadata using NewSQL databases," in *Proc. of USENIX FAST*, 2017.
- [47] S. Pan, T. Stavrinou, Y. Zhang, A. Sikaria, P. Zakharov, A. Sharma, M. Shuey, R. Wareing, M. Gangapuram, G. Cao, C. Preseau, P. Singh, K. Patiejunas, J. Tipton, E. Katz-Bassett, and W. Lloyd, "Facebook's Tectonic filesystem: Efficiency from exascale," in *Proc. of USENIX FAST*, 2021.
- [48] S. Patil and G. Gibson, "Scale and concurrency of GIGA+: File system directories with millions of files," in *Proc. of USENIX FAST*, 2011.
- [49] K. Ren, Q. Zheng, S. Patil, and G. Gibson, "IndexFS: Scaling file system metadata performance with stateless caching and bulk insertion," in *Proc. of IEEE SC*, 2014.
- [50] D. Roselli, J. R. Lorch, and T. E. Anderson, "A comparison of file system workloads," in *Proc. of USENIX ATC*, 2000.
- [51] M. A. Sevilla, N. Watkins, C. Maltzahn, I. Nassi, S. A. Brandt, S. A. Weil, G. Farnum, and S. Fineberg, "Mantle: A programmable metadata load balancer for the ceph file system," in *Proc. of IEEE SC*, 2015.
- [52] S. Sheng, J. Cai, Q. Huang, L. Tang, and P. P. Lee, "Toward distributed write-back caching in programmable switches," *IEEE Trans. on Networking*, vol. 33, no. 5, pp. 2569–2584, October 2025.
- [53] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop distributed file system," in *Proc. of IEEE MSST*, 2010.
- [54] A. Thomson and D. J. Abadi, "CalvinFS: Consistent WAN replication and scalable metadata management for distributed file systems," in *Proc. of USENIX FAST*, 2015.
- [55] F. Wang, J. Qiu, J. Yang, B. Dong, X. Li, and Y. Li, "Hadoop high availability through metadata replication," in *Proc. of ACM CIKM*, 2009.
- [56] Q. Wang, Y. Lu, E. Xu, J. Li, Y. Chen, and J. Shu, "Concordia: Distributed shared memory with in-network cache coherence," in *Proc. of USENIX FAST*, 2021.
- [57] Y. Wang, C. Li, X. Shao, Y. Chen, F. Yan, and Y. Xu, "Lunule: An agile and judicious metadata load balancer for CephFS," in *Proc. of IEEE SC*, 2021.
- [58] Y. Wang, Y. Wu, C. Li, P. Zheng, B. Cao, Y. Sun, F. Zhou, Y. Xu, Y. Wang, and G. Xie, "CFS: Scaling metadata service for distributed file system via pruned scope of critical sections," in *Proc. of ACM EuroSys*, 2023.
- [59] S. Weil, S. A. Brandt, E. L. Miller, D. D. Long, and C. Maltzahn, "Ceph: A scalable, high-performance distributed file system," in *Proc. of USENIX OSDI*, 2006.
- [60] B. Welch, M. Unangst, Z. Abbasi, G. A. Gibson, B. Mueller, J. Small, J. Zelenka, and B. Zhou, "Scalable performance of the Panasas parallel file system," in *Proc. of USENIX FAST*, 2008.
- [61] L. Xiao, K. Ren, Q. Zheng, and G. A. Gibson, "ShardFS vs. IndexFS: Replication vs. caching strategies for distributed metadata management in cloud storage systems," in *Proc. of ACM SoCC*, 2015.
- [62] J. Xu, M. Dong, Q. Tian, Z. Tian, T. Xin, and H. Chen, "SwitchFS: Asynchronous metadata updates for distributed filesystems with in-network coordination," in *Proc. of ACM EuroSys*, 2026.
- [63] Z. Yu, Y. Zhang, V. Braverman, M. Chowdhury, and X. Jin, "NetLock: Fast, centralized lock management using programmable switches," in *Proc. of ACM SIGCOMM*, 2020.
- [64] H. Zhang, K. Cheng, R. Chen, and H. Chen, "Fast and scalable in-network lock management using lock fission," in *Proc. of USENIX OSDI*, 2024.
- [65] B. Zhao, W. Wu, and W. Xu, "NetRPC: Enabling in-network computation in remote procedure calls," in *Proc. of USENIX NSDI*, 2023.
- [66] H. Zhu, Z. Bai, J. Li, E. Michael, D. Ports, I. Stoica, and X. Jin, "Harmonia: Near-linear scalability for replicated storage with in-network conflict detection," *Proc. of the VLDB Endowment*, vol. 13, no. 3, pp. 376–389, 2019.

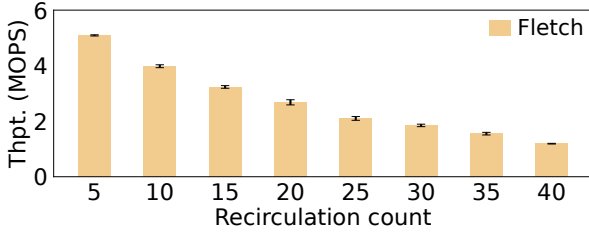


Figure 16: (Exp#S1) Switch processing throughput of Fletch under high recirculation counts.

APPENDIX

We report two additional experiments: (i) the recirculation overhead under higher write ratios (Exp#S1) and (ii) system recovery time (Exp#S2).

ADDITIONAL EXPERIMENTS

(Exp#S1) Switch processing throughput of Fletch under high recirculation counts. We evaluate the switch’s processing capacity under recirculation counts beyond the typical range observed in real-world workloads (Exp#1). We focus on Fletch in this experiment. We vary the per-request recirculation count from 5 to 40. The upper bound of 40 represents the worst case under high write ratios: Exp#3 mixes read and write requests across the full ratio spectrum (0% to 100% *chmod*) and observes a peak recirculation count of 39.42 at 50% *chmod*, beyond which the recirculation count decreases as fewer concurrent reads contend with writes for locks.

Following the methodology of switch overhead analysis in Exp#1, we pre-load cached paths into the in-switch cache and use a DPDK-based client [8] to issue *stat* requests exclusively on these paths, so that all requests are served entirely by the switch. As illustrated in Exp#1, a cache-hit *stat* request at depth L incurs $L + 2$ recirculations. For a maximum path depth of 9 (our evaluation default), the natural recirculation count is at most 11, far below our target of 40. To reach higher recirculation counts, we extend the Fletch’s switch to accept a target recirculation count r as a per-request input. Before the *stat* request on a path at depth L enters the normal in-switch caching pipeline, the switch performs $r - (L + 2)$ extra recirculations with no caching logic; after these extra recirculations, the request follows the normal in-switch caching processing. We select a cached path at depth 3 across all r values. For each recirculation count r , we increase the client’s sending rate until the switch begins dropping packets internally and report the highest sustained throughput.

Figure 16 shows the switch throughput across recirculation counts. The throughput decreases gradually from 5.1 MOPS at the recirculation count of 5 to 1.2 MOPS at the recirculation count of 40, but never collapses. Even at the recirculation count of 40, the switch throughput remains $4.0\times$ higher than the aggregate throughput of NoCache on 16 simulated servers (around 0.3 MOPS in Exp#3). Thus, recirculation does not cause data-plane congestion on 16 simulated servers, even under recirculation counts (e.g., 40) far beyond those observed in real-world workloads (at most 5.61 in Exp#1).

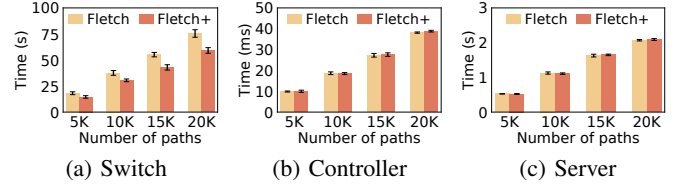


Figure 17: (Exp#S2) System recovery time.

We acknowledge that under write-intensive workloads, the recirculation overhead becomes significant. How to mitigate the recirculation overhead in Fletch is posed as future work.

(Exp#S2) System recovery time. We evaluate the crash recovery time of Fletch and Fletch+ under switch, controller, and server failures for varying numbers of pre-loaded paths (from 5K to 20K). We first pre-load a given number of paths by triggering cache admission, which populates the in-switch cache, the server’s path-token map, and the controller’s path-token and hash-token maps (§VI-A). We then manually kill a target component to mimic a failure, and trigger the recovery procedure (§VII-C). For controller and server failures, the restarted component passes a recovery flag at startup: the controller reconstructs its maps from its local historical log, while the server queries the controller for its path-token entries in the active log. For a switch failure, we reboot the switch to reset its data-plane state. We run a script to replay cache admission for each path in the active log, so that the controller re-fetches the metadata and re-installs both the metadata and the hash-token map into the switch, with each path retaining its original token.

Figure 17 shows the recovery time for each component. The recovery time grows roughly linearly with the number of paths, as each path is processed sequentially. From 5K to 20K paths, server recovery takes 0.5 s to 2.1 s, controller recovery takes 9.9 ms to 38.8 ms, and switch recovery takes 14.6 s to 75.5 s. Switch recovery is the slowest as it replays cache admission for each path, which involves fetching metadata from the backend and installing it along with the hash-token map into the switch. Controller recovery is the fastest as it only reads the local historical log without network communication. Server recovery falls in between, as the controller transmits the active log via UDP, incurring network round-trips.

Fletch+ recovers 21.6% faster than Fletch on switch recovery at 20K paths, because switch recovery requires fetching metadata from the backend, and RocksDB (Fletch+’s backend) provides higher throughput than HDFS (Fletch’s backend). For controller and server recovery, Fletch and Fletch+ have nearly identical times, as both procedures involve only local log reads or UDP transfers without metadata fetching.